

Docker Notes	3
Virtualization	3
Contanerization:	3
Docker	3
Docker Installation:	3
Basic Docker Commands	3
Docker Images	4
Docker Volumes	8
Docker Networking	8
Containers	9
Eg:1 Mounting a Local Directory	11
Eg:2 Mounting a File	12
Eg:3 Using Docker Volumes	12
Why Use Volumes?	12
To Login to Container:	12
Docker Logs & Monitoring	13
To Commit Docker container changes as new image	13
Inspect Docker resources	13
DockerFiles	14
Example 1: Ubuntu Container	14
Example 2: Using WORKDIR and COPY	14
Example 3: Environment Variables	14
Example 4: ENTRYPOINT vs CMD	14
DockerFile Commands	15
Docker-Compose	16
Docker-Compose Installation	16
Docker-Compose File-1	16
Docker-Compose File-2	17

Docker-Compose Commands	18
Docker-Compose File3	19
Docker-Compose-File4: ubuntu container with volumes	19
Docker-Compose-File5: Volume managed by docker	20
Docker-Compose-File6: with network and volumes on composefile	20
Docker Swarm	22
Docker Swarm - yaml File	24

Docker Notes

Virtualization

It is a technology that creates virtual versions of the computer resources such as hardware resources, operating system, storage devices, network resources. It is likely to create a software based replica of a physical machine allowing you to run multiple isolated environments on the same hardware or across a distributed system.

Containerization:

It is a light-weight form of virtualization that allows to run application and their dependencies in isolated containers. Each container shares the same operating system kernel but it is isolated from other containers.

Docker

Docker is an open-source platform that lets you build, package, and run applications inside containers. Containers are lightweight, portable, and ensure your app runs the same way across different environments

Docker Installation:

Installing Docker on Amazon Linux (Amazon Linux 2 or Amazon Linux 2023, ...)

- `#sudo yum update -y`
- `#sudo yum install docker -y` ==> Docker installation
- `#service docker status` ==> Check Docker Service is running or not
- `#systemctl status docker`
- `#service docker start`
- `#systemctl start docker`

Basic Docker Commands

- `#docker --version` ==> Check Docker version
- `#docker -v` ==> Check Docker version
- `#docker info` ==> Display system-wide information
- `#docker help` ==> Show help for Docker commands
- `#docker login` ==> login to docker hub account

Docker Images

- `#docker images` ==> To list images present
 - `#docker image ls` ==> To list images present
 - `#docker pull <Image_Name>:<Tag>` ==> Download image from Docker Hub
- Eg:** `#docker pull nginx:latest`
- Eg:** `#docker pull ubuntu:20.04`
- `#docker history <Image_Name>:<Tag>` ==> Shows layer history with size
 - `#docker inspect <Image_Name>:<Tag>` ==> Detailed JSON metadata
 - `#docker build -t <Image Name>:<Tag> .` ==> To build image from Docker file

Eg: `#docker build -t yndurgarao1/image1:0.1 .`

Explanation:

`.(dot)` ==> Represents current working directory where Docker file present

`-t` ==> Tag image with Image Name:tag

- `#docker build -f <Docker_File_Name> -t <Image_Name>:<Tag> .`

Eg: `#docker build -f Dockerfile1 -t yndurgarao1/myimage:0.1 .`

Explanation:

`-f Dockerfile1` ==> Specifies the Docker file to use. By default, Docker looks for Docker file, but here you're explicitly pointing to Dockerfile1

`-t yndurgarao1/myimage:0.1` ==> Tags the image with a name and version

`yndurgarao1` ==> it is the repository/name space (your Docker Hub username)

`myimage` → image name

`0.1` → image tag version

- `#docker build --no-cache -t <Image_Name>:<Tag> .`

Explanation:

`--no-cache` ==> Ignores all cached layers and rebuilds everything fresh.
Useful when debugging Dockerfile changes or
ensuring you're not reusing stale layers.

- **Docker File to check latest base image version**

DockerFile

```
FROM ubuntu:22.04

RUN apt-get update && apt-get install -y curl

CMD ["bash"]
```

`#docker build --pull -t <Image_Name>:<Tag> .`

Explanation

`docker build` ==> Builds a Docker image from a Dockerfile.

`--pull` ==> Forces Docker to always check and pull the latest
version of the base image (e.g., `ubuntu:22.04`)
even if a cached copy exists locally.

`-t <Image_Name>:<Tag>` ==> Tags the resulting image with a name and
version. Example: `myapp:1.0`.

`.` ==> Build context is the current directory (where the
Docker file and related files are located)

- **Dockerfile example using ARG for build-time arguments, which you can pass when running docker build**

DockerFile

```
FROM ubuntu:22.04                # Base image
ARG APP_VERSION                  # Declare build-time arguments
ARG APP_ENV=production           # default value
RUN echo "Application Version: $APP_VERSION" # Use ARG inside build steps
RUN echo "Environment: $APP_ENV"          # Use ARG inside build steps
ENV VERSION=$APP_VERSION             # Promote ARG to ENV if needed at runtime
ENV ENVIRONMENT=$APP_ENV             # Promote ARG to ENV if needed at runtime
CMD ["bash"]                        # Example command
```

Pass arguments explicitly

```
#docker build --build-arg APP_VERSION=1.0 --build-arg APP_ENV=dev -t myapp:1.0 .
```

Use default for APP_ENV

```
docker build --build-arg APP_VERSION=2.0 -t myapp:2.0 .
```

- `#docker build <Repo_URL>#<Branch> -t <Image_Name>:<Tag>`

Explanation:

branch:dir ==> optional fragment specifying

branch ==> which branch to use (e.g., main, dev).

Dir ==> subdirectory inside the repo containing the Dockerfile.

Eg:1 `#docker build https://github.com/yndurgarao1/docker_ynd.git#main -t myimage466:0.1`

Repo_URL: https://github.com/yndurgarao1/docker_ynd.git

Branch: main

Eg:2 `#docker build`

`https://github.com/yndurgarao1/docker_ynd.git#docker:Docke/Docke_File -t image_folder:0.1`

Docke/Docke_File ==> folder in the git path for DockerFile

- `#docker rmi <Image Name>:<tag>` ==> Remove image

Eg: `#docker rmi ubuntu:20.04`

Eg: `#docker rmi nginx`

- `#docker rmi $(docker images -q)` ==> To remove unused images

`docker images -q` ==> lists all image IDs on the host (quiet mode).

- `#docker push <Image_Name>:<Tag>` ==> To push image to DockerHub Registry

Eg: `#docker push my_image:0.1`

Docker Volumes

- `#docker volume ls` ==> To list volumes
- `#docker volume create <Volume_Name>` ==> To create volume
Eg: `#docker volume create volume1`
- `#docker volume inspect <Volume_Name>` ==> Inspect volume details
Eg: `#docker volume inspect volume1`
- `#docker volume rm <Volume_Name>` ==> To remove volume
Eg: `#docker volume rm volume1`
- `#docker volume prune`
Removes all dangling volumes (volumes not attached to any container).
Prompts for confirmation unless you add `-f` (force).

Docker Networking

- Bridge network ==> default, single host communication.
- Overlay network ==> multi-host network used in Docker swarm.
- `#docker network ls` ==> To list networks
- `#docker network create <Network Name>` ==> To create new network
Eg: `#docker network create Network1`
- `#docker network inspect <Network Name>` ==> To inspect network details
Eg: `#docker network inspect Network1`
- `#docker network rm <Network Name>` ==> To remove network
Eg: `#docker network rm network1`
- `#docker network prune` ==> To removes unused network

Containers

- `#docker ps` ==> To list running containers
- `#docker ps -a` ==> To list all containers (including stopped, paused, running)
- `#docker run hello-world` ==> To create docker container

Explanation:

`docker run` ==> To starts a new container.

`hello-world` ==> Image name. If it's not already on your system, Docker will automatically pull it from Docker Hub.

- `#docker run -it <Image Name> /bin/bash` ==> Run container interactively

Eg: `#docker run -it nginx /bin/bash`

Note: Cont ==> Container

- `#docker start <Cont_Name or Cont_ID>` ==> To start a stopped container
- `#docker stop <Contr_Name or Cont_ID>` ==> To stop a running container
- `#docker pause <Cont_Name or Cont_ID>` ==> To Pause running container
- `#docker unpause <Cont_Name or Cont_ID>` ==> Unpause the paused container
- `#docker rm <Cont_Name or Cont_ID>` ==> To remove the stopped container
- `#docker rm $(docker ps -aq)` ==> Removes all stopped containers

- `#docker run -dit --name <Container name> <Image_Name>:<tag>`

Explanation:

- | | | |
|---|---------------------|--|
| <code>-d (detached mode)</code> | <code>==></code> | Runs the container in the background. You won't see logs directly in your terminal, but the container keeps running. |
| <code>-i (interactive mode)</code> | <code>==></code> | Keeps STDIN open, even if not attached. Useful when you want to interact with the container later. |
| <code>-t (pseudo-TTY)</code> | <code>==></code> | Allocates a terminal inside the container. This makes it possible to run commands interactively. |
| <code>--name <Container name></code> | <code>==></code> | Assigns a custom name to the container. |
| <code><Image name>:<tag></code> | <code>==></code> | Specifies the image and version (tag) to use. |

Eg: `#docker run -dit --name my_cont ubuntu:20.04`

- `#docker run -dit --name <Cont_Name> -p <Host_Port>:<Cont_Port> <Image_Name>:<Tag>`

Explanation:

- | | | |
|------------------------|---------------------|--|
| <code>-p</code> | <code>==></code> | Maps a port on your host machine to a port inside the container. |
| <code>Host_Port</code> | <code>==></code> | The port you'll use on your local machine. |
| <code>Cont_Port</code> | <code>==></code> | The port the application inside the container listens on. |

- `#docker run -dit --name <Cont_Name> -v <Host_Path>:<Cont_Path> <Image_name>:<Tag>`

Explanation:

`-v` ==> Mounts a directory or file from your host machine into the container.

`Host_Path` ==> Path on your local machine.

`Cont_Path` ==> Path inside the container where it will be accessible.

- `#docker run --rm alpine echo "Hello Kadali"` ==> Run and autoremove after execution

- `#docker run -dit --name cont1 --network ynd_network -v /home/ec2-user/durga:/renuka -p 8866:80 nginx:latest`

- `#docker run -dit --name cont1 --network network1 ubuntu:latest`

Explanation:

`docker run` ==> Starts a new container.

`-dit` ==> Runs in detached mode (-d), keeps STDIN open (-i), and allocates a terminal (-t).

`--name cont1` ==> Names the container cont1.

`-network network1` ==> Connects the container to a user-defined Docker network called network1.

`ubuntu:latest` ==> Uses the latest Ubuntu image from Docker Hub

Eg:1 Mounting a Local Directory

- `#docker run -dit --name my_nginx -p 8080:80 -v /home/user/html:/usr/share/nginx/html nginx:latest`

Explanation:

❖ Maps host port 8080 ==> Container port 80.

❖ Mounts local directory /home/user/html into the container at /usr/share/nginx/html.

❖ Any changes you make in /home/user/html on your host will reflect inside the container.

Eg:2 Mounting a File

- `#docker run -dit --name my_app -v /home/user/config.json:/app/config.json my_image:latest`
 - ❖ Mounts a single file **config.json** from the host system into the container at **/app/config.json**

Eg:3 Using Docker Volumes

Instead of host paths, you can use named volumes:

- `#docker run -dit --name my_db -v my_volume:/var/lib/mysql mysql:8.0`
 - ❖ Creates or uses a Docker-managed volume named **my_volume**.
 - ❖ Mounts it into the container at **/var/lib/mysql** for persistent database storage.

Why Use Volumes?

- **Persistence:** Data survives container restarts/removals.
- **Sharing:** Multiple containers can share the same data.
- **Flexibility:** Easily swap configs or code without rebuilding images.

To Login to Container:

- `#docker exec -it <Cont_Name> /bin/bash`

Explanation:

`docker exec` ==> Executes a command inside an already running container.

`-i` ==> Keeps STDIN open (interactive).

`-t` ==> Allocates a pseudo-terminal (TTY).

`/bin/bash` ==> The command to run inside the container. In this case, it starts a Bash shell.

Docker Logs & Monitoring

- `#docker logs <Cont_Name or Cont_ID>` ==> To view container logs
- `#docker logs -f <Cont_Name or Cont_ID>` ==> To view logs
- `#docker logs -t <Cont_Name or Cont_ID>` ==> to view logs with time stamp
- `#docker logs --tail 100 <Cont_Name or Cont_ID>`
- `#docker logs --since 10m <Cont_Name or Cont_ID>`
- `#docker logs --until 2026-04-23T08:00:00 <Cont_Name or Cont_ID>`
- `#docker stats <Cont_Name or Cont_ID>` ==> Show resource usage of containers
- `#docker inspect <Cont_Name or Cont_ID>` ==> Detailed info about container
- `#docker stats` ==> to see real time container statistics
- `#docker stats --no-stream`
- `#docker top <Cont_Name Or Cont_ID>`

To Commit Docker container changes as new image

I created a container and done some changes in top of that container I want to reuse that change when I am building a new container.

- `#docker commit <Cont_Name> <New_Image_name>:<Tag>`
- `#docker commit -a <author name> -m <commt message> <Container Name> <New Image_Name>:<Tag>`

Inspect Docker resources

- `#docker inspect <Cont_Name>` ==> To Inspect docker container
- `#docker inspect <Image_Name>` ==> To Inspect docker image
- `#docker inspect <Volume_Name>` ==> To inspect docker volume
- `#docker inspect <Network_Name>` ==> To inspect docker network

DockerFiles

Example 1: Ubuntu Container

```
FROM ubuntu:20.04          # Start from Ubuntu base image
RUN apt-get update && apt-get install -y curl    # Run update and install curl
CMD ["echo", "Hello from Ubuntu container!"]    # Default command
```

Example 2: Using WORKDIR and COPY

```
FROM python:3.9            # Start from Python base image
WORKDIR /app               # Set working directory
COPY . /app                # Copy local files into container
RUN pip install -r requirements.txt # Install dependencies
CMD ["python", "app.py"]    # Run application
```

Example 3: Environment Variables

```
FROM node:16
WORKDIR /usr/src/app
COPY package*.json ./
RUN npm install
COPY . .
ENV PORT=3000      # Set environment variable
EXPOSE 3000
CMD ["node", "server.js"]
```

Example 4: ENTRYPOINT vs CMD

```
FROM alpine:latest
RUN apk add --no-cache curl # Install curl
ENTRYPOINT ["curl"] # Define entrypoint
CMD ["--help"]      # CMD provides default arguments
```

DockerFile Commands

- FROM ==> base image
- RUN ==> execute commands during build
- WORKDIR ==> set working directory
- COPY ==> copy files into container
- ENV ==> set environment variables
- CMD ==> default command
- ENTRYPOINT ==> fixed executable

Difference Between CMD and ENTRYPOINT:

The key difference is that CMD provides default commands or arguments that can be fully overridden at runtime, while ENTRYPOINT defines the main executable that always runs, with runtime arguments only appended. The key difference is that CMD provides default commands or arguments that can be fully overridden at runtime, while ENTRYPOINT defines the main executable that always runs, with runtime arguments only appended.

CMD vs ENTRYPOINT in Docker

Aspect	CMD	ENTRYPOINT
Purpose	Sets default command/arguments for the container	Defines the fixed executable the container always runs
Runtime Override	Entirely replaced if you pass a command in <code>docker run</code>	Stays fixed; only arguments can be changed unless you use <code>--entrypoint</code>
Flexibility	Very flexible, good for general-purpose images	More rigid, ensures container behaves like a dedicated app
Typical Use Case	Provide defaults (e.g., <code>bash</code> , <code>nginx</code> , <code>-g 'daemon off;'</code>)	Force container to run a specific program (e.g., <code>python3</code> , <code>nginx</code>)
Combination	Often paired with ENTRYPOINT to supply default args	ENTRYPOINT runs the program, CMD supplies default args

Docker-Compose

Docker Compose is a tool for defining and running multi-container Docker applications. Instead of manually starting containers one by one, you describe your entire stack in a YAML file (docker-compose.yml) and bring it up with a single command.

Docker-Compose Installation

- Download the latest binary:

```
# sudo curl -L "https://github.com/docker/compose/releases/download/v2.27.0/docker-compose-$(uname -s)-$(uname -m)" -o /usr/local/bin/docker-compose
```

- Apply executable permissions:

```
#sudo chmod +x /usr/local/bin/docker-compose
```

- Verify:

```
#docker-compose --version
```

Docker-Compose File-1

```
version: "3.9"
services:
  ubuntu:
    image: ubuntu:22.04
    container_name: ubuntu_container
    command: sleep infinity # Keeps container alive
    tty: true              # Allows interactive terminal sessions
    restart: always
```

```
#docker-compose -f Docker-Compose.yaml up -d
```

Here Docker-Compose.yaml ==> Docker-Compose yaml file

image: ubuntu:22.04 ==> Pulls the official Ubuntu 22.04 image.

command: sleep infinity ==> Prevents the container from exiting immediately.

tty: true ==> Lets you attach an interactive shell (docker compose exec ubuntu bash).

restart: always ==> Ensures the container restarts automatically if stopped.

Docker-Compose File-2

```
services:
  ubuntu:
    image: ubuntu:22.04
    container_name: ubuntu_service
    command: sleep infinity
    tty: true
    volumes:
      - ubuntu_data:/data    # Named volume
    networks:
      - mynet
  nginx:
    image: nginx:latest
    container_name: nginx_service
    ports:
      - "8080:80"
    volumes:
      - web_content:/usr/share/nginx/html # Named volume
    networks:
      - mynet
volumes:
  ubuntu_data:  # Docker-managed volume
  web_content:  # Docker-managed volume
networks:
  mynet:
    driver: bridge
```

```
#docker-compose -f Docker-Compose-File2 up -d
```

Docker-Compose Commands

Step	Command	Purpose
Start services	<code>docker compose up -d</code>	Launch containers in background
Stop services	<code>docker compose down</code>	Stop and remove containers, networks
Scale services	<code>docker compose up --scale web=3 -d</code>	Scale replicas on one host
View logs	<code>docker compose logs -f web</code>	Tail logs for a service
Check status	<code>docker compose ps</code>	Show running containers
Rebuild images	<code>docker compose build</code>	Rebuild images defined in YAML
Validate file	<code>docker compose config -q</code>	Check YAML syntax and validity

- `#docker-compose config --services ==>` list down the services mentioned in docker-compose.yml file
- `#docker-compose up -d`
 - `docker-compose ==>` Uses the docker-compose.yml file in the current directory to define services, networks, and volumes.
 - `up ==>` Builds, (re)creates, starts, and attaches to containers defined in the Compose file.
 - `-d (detached mode) ==>` Runs containers in the background, freeing your terminal.
- `#docker-compose down`
 - Stops containers defined in docker-compose.yml.
 - Removes containers, networks, and default volumes created by docker-compose up.
 - Leaves images intact (unless you specify otherwise).
- `#docker-compose up -d <New_Service_Name> ==>` Modified yaml on new service
- `#docker-compose ps ==>` List running containers of docker-compose.yml
- `#docker-compose ps <Service_name>`

- #docker-compose stop
- #docker-compose restart
- #docker-compose logs
- #docker-compose pause
- #docker-compose unpause
- #docker-compose ls

Docker-Compose File3

#cat docker-compose.yml

```
services:
  ubuntu:
    image: ubuntu:22.04
    container_name: ubuntu_container
    command: sleep infinity # Keeps container alive
    tty: true              # Allows interactive terminal sessions
    restart: always
```

Explanation

image: ubuntu:22.04	==> Pulls the official Ubuntu 22.04 image.
command: sleep infinity	==> Prevents the container from exiting immediately.
tty: true	==> Lets you attach an interactive shell (docker compose exec ubuntu bash).
restart: always	==> Ensures the container restarts automatically if stopped.

- # docker-compose config --services ==> Prints the names of all services defined in the Compose file

Docker-Compose-File4: ubuntu container with volumes

cat docker-compose-example1.yml

```
services:
  Base_Image:
    image: ubuntu:latest
    container_name: Ubuntu_Container
    command: sleep infinity # Keeps container alive
    tty: true              # Allows interactive terminal sessions
    restart: always
    volumes:
      - /root/durga:/ynd
```

➤ # docker-compose -f Docker-Compose-File4.yml up -d

Explanation:

-f docker-compose-example1.yml ==> Use a specific Compose file named Docker-Compose-File4.yml instead of the default docker-compose.yml.

Docker-Compose-File5: Volume managed by docker

```
services:
  ubuntu:
    image: ubuntu:22.04
    container_name: ubuntu_volume_example
    command: tail -f /dev/null # keeps container running
    volumes:
      - mydata:/app/data # named volume → container path
volumes:
  mydata: # Docker manages this volume
```

Docker-Compose-File6: with network and volumes on composefile

[root@ip-172-31-81-229 ec2-user]# cat docker-compose.yml

```
networks:
  ynd_network:
    driver: bridge
volumes:
  ynd_volume:
services:
  web:
    container_name: nginx_container
    image: nginx:latest
    networks:
      - ynd_network
    volumes:
      - ynd_volume:/durga
```

- `#docker-compose up -d --build <Service_Name> ==>` to remodify existing service
- `#docker-compose config --services ==>` to list services under docker-compose.yaml file
- `#docker-compose down ==>` to stop and remove containers under docker-compose.yaml file
- `#docker-compose stop <Service_Name> ==>` to stop specific service
- `#docker-compose rm <Service_Name> ==>` to remove specific service container in docker-compose.yaml
- `# docker-compose -f docker-compose-example1.yml ps ==>` list running containers
- `# docker-compose -f docker-compose-example1.yml logs -f ==>` To View logs
- `# docker-compose -f docker-compose-example1.yml down ==>` Stop and clean up
- `# docker-compose -f docker-compose-example1.yml up -d --build ==>` if you've changed Dockerfiles or dependencies, use:
- `# docker-compose down -v ==>` Also remove volumes.
- `# docker-compose down --rmi all ==>` Remove images built by Compose
- `# docker-compose logs ==>` View logs from all services.
- `# docker-compose logs -f ==>` Follow logs live.
- `# docker-compose start ==>` Start existing containers.
- `# docker-compose stop ==>` Stop services without removing containers.
- `# docker-compose restart ==>` Restart services.
- `# docker-compose run <service> <command> ==>` Run a one-off command in a new container.
- `# docker-compose build ==>` Build or rebuild service images.
- `# docker-compose pull ==>` Pull service images from registry.
- `# docker-compose push ==>` Push built images to registry.
- `# docker-compose images ==>` List images used by services.

Docker Swarm

Command	Purpose	Example
<code>docker swarm init</code>	Initialize a new swarm (manager node)	<code>docker swarm init --advertise-addr 192.168.1.10</code>
<code>docker swarm join</code>	Add a node to swarm	<code>docker swarm join --token 192.168.1.10:2377</code>
<code>docker swarm join-token</code>	Show/rotate join tokens	<code>docker swarm join-token worker</code>
<code>docker swarm leave</code>	Remove node from swarm	<code>docker swarm leave --force</code>
<code>docker swarm update</code>	Update swarm settings	<code>docker swarm update --dispatcher-heartbeat 10s</code>
<code>docker swarm ca</code>	Manage root CA	<code>docker swarm ca --rotate</code>
<code>docker swarm unlock</code>	Unlock a locked swarm	<code>docker swarm unlock</code>
<code>docker swarm unlock-key</code>	Manage unlock keys	<code>docker swarm unlock-key rotate</code>

Command	Purpose	Example Usage
<code>docker service create</code>	Deploy a new service in the swarm	<code>docker service create --name web --replicas 3 -p 80:80 nginx</code>
<code>docker service ls</code>	List all services running in the swarm	<code>docker service ls</code>
<code>docker service inspect</code>	Detailed info about a service (JSON output)	<code>docker service inspect web</code>
<code>docker service ps</code>	Show tasks (containers) of a service	<code>docker service ps web</code>
<code>docker service scale =N</code>	Scale service to N replicas	<code>docker service scale web=5</code>
<code>docker service update</code>	Modify service (image, replicas, env vars, etc.)	<code>docker service update --image nginx:latest web</code>
<code>docker service logs</code>	View logs of a service	<code>docker service logs web</code>
<code>docker service rm</code>	Remove a service from the swarm	<code>docker service rm web</code>

- `#docker node ls ==>` It lists all nodes participating in the swarm, showing their role, availability, and status.

- `#docker swarm join-token worker ==>` to get the worker token
- `#docker swarm join-token manager ==>` to get manager token
- `#docker swarm leave ==>` to remove a node from a Docker Swarm cluster. [in worker node we need to execute]
- `#docker node demote <node_id> ==>` Node becomes a worker and no longer participates in Raft [need to be executed in manager node]
- `#docker node rm <node_id> ==>` to remove the worker node [we need to execute in manager node]
- `#docker service create --name <Service_Name> --replicas <Number> -p Container_Port: Actual_Port <Image_Name>:<Tag>`

Eg: #docker service create --name my_service --replicas 4 -p 8800:80 nginx:latest

`docker service create ==>` Deploys a new service in the swarm.

`--name nginx_service ==>` Names the service.

`--replicas 5 ==>` Ensures 5 identical containers (tasks) are running across the swarm cluster.

`-p 8080:80 ==>` Maps host port 8080 to container port 80.

`nginx:latest ==>` Uses the official Nginx image.

- `#docker service ls ==>` to list the service name
 - `#docker service ps <Service_name> ==>` to list running services information
 - `#docker service scale <Service_Name>=<Number> ==>` to scale up or down
- Eg: #docker service scale my_service=7**
- `#docker service inspect <Service_Name> ==>` to get more info about service
 - `#docker service update --image nginx:alpine <Service_name> ==>` to update existing service by latest image
 - `#docker service rm <Service_name> ==>` to remove the service
 - `#docker node promote <node_id> ==>` to promote as manager
 - `#docker node demote <node_id> ==>` to demote as worker from manager or reachable
 - `#docker service update --rollback <service_name> ==>` it will roll back to previous image
 - `#docker service update --force <Service_Name> ==>` redeploys without spec change.

Docker Swarm - yaml File

```
services:
  web:
    image: nginx:latest
    ports:
      - "8080:80"
    deploy:
      replicas: 3
    placement:
      constraints: [node.role == worker]
    networks:
      - frontend
    volumes:
      - web_content:/usr/share/nginx/html
  db:
    image: mysql:5.7
    environment:
      MYSQL_ROOT_PASSWORD: example
    deploy:
      replicas: 1
    placement:
      constraints: [node.role == manager]
    networks:
      - backend
    volumes:
      - db_data:/var/lib/mysql
networks:
  frontend:
    driver: overlay
  backend:
    driver: overlay
volumes:
  web_content:
  db_data:
```


Command	Purpose	Best Example
<code>docker stack deploy</code>	Deploy a stack from a Compose file	<code>docker stack deploy -c docker-compose.yml mystack</code>
<code>docker stack ls</code>	List all stacks	<code>docker stack ls</code>
<code>docker stack services</code>	List services in a stack	<code>docker stack services mystack</code>
<code>docker stack ps</code>	Show tasks (containers) in a stack	<code>docker stack ps mystack</code>
<code>docker stack rm</code>	Remove a stack	<code>docker stack rm mystack</code>

- `#docker stack deploy -c docker-compose.yml stack1`
`docker stack deploy` ==> Deploys a stack (multi- service application) into Docker Swarm.
`-c docker-compose.yml` ==> Specifies the Compose file that defines services
`mystack` ==> The name of the stack; all services will be grouped under this name.
- `#docker stack ls` ==> it will display the information about stacks present
- `#docker stack services <Stack_Name>`
- `#docker stack ps <Stack_Name>`
- `#docker stack rm mystack`
- `#docker service scale <stack_name>_<Service_name>=<Scaling number>`
Eg: `#docker service scale mystack1_srv1=6`